Studio Nx

JS 115 — ticksToBars()

2026-10-14

JS 115 — `ticksToBars()` (tick을 마디로 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js` (편곡 영역 모듈에 추가)
작업	S1-06 타임라인: 첫 번째 함수 (FR-209 좌표 기준)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline` (`develop`에서 만듦)
다음	이 함수의 역방향 `barsToTicks()`와 스냅 단위 함수를 한 함수씩 이어서 진행합니다

1. 하는 일

엔진·DB의 **tick**(1박 = 960)을 편곡 화면의 **마디**로 바꿉니다. 편곡 화면은 마디 단위로 그리고, 소리 엔진과 DB는 tick 단위를 쓰기 때문에 두 단위를 잇는 변환이 필요합니다.

> 마디 = tick ÷ (한 마디의 tick 수) 예: 3840 tick ÷ 3840 = 1마디

입력	결과
0 tick	0마디
960 tick (1박)	0.25마디
480 tick (1/8박)	0.125마디
3840 tick (4/4 한 마디)	1마디
15360 tick (4마디)	4마디
-960 tick (카운트인처럼 0보다 앞)	-0.25마디
두 번째 인자 4800 tick (5/4 한 마디)	1마디
tick이 NaN·Infinity·문자열, 마디 길이가 0 이하	오류(`RangeError`)

2. 코드

```
import { PPQ } from '../audio/timing.js';

export const TICKS_PER_BAR = PPQ * 4;

export function ticksToBars(tick, ticksPerBar = TICKS_PER_BAR) {
  if (typeof tick !== 'number' || !isFinite(tick)) throw new RangeError('tick 은 유한한 숫자여야 합니다:
' + tick);
  if (typeof ticksPerBar !== 'number' || !isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeErro
r('ticksPerBar 는 0 보다 큰 숫자여야 합니다: ' + ticksPerBar);
  return tick / ticksPerBar;
}
```

최신 문법으로 쓰면 (공부용)

```
export function ticksToBars(tick, ticksPerBar = TICKS_PER_BAR) {
  if (!Number.isFinite(tick)) throw new RangeError(`tick 은 유한한 숫자여야 합니다: ${tick}`);
  if (!Number.isFinite(ticksPerBar) || ticksPerBar <= 0) throw new RangeError(`ticksPerBar 는 0 보다 큰 숫자여야 합니다: ${ticksPerBar}`);
  return tick / ticksPerBar;
}
```

최신 문법	실제 코드	뜻
`Number.isFinite(x)`	`typeof x !== 'number' || !isFinite(x)`	숫자이면서 NaN·Infinity가 아닌지 확인합니다
템플릿 문자열 `` `…${x}` ``	`'…' + x`	글자와 값을 잇습니다
기본값 인자	그대로 사용	두 번째 인자를 안 주면 4/4(3840)를 씁니다

3. 왜 이렇게

선택	이유
tick → 마디 변환을 먼저 만듦	편곡 화면은 마디, 엔진·DB는 tick을 씁니다. S1-05 재생선 위치(`Transport.positionTick()`)를 마디로 바꿔 넘길 때 가장 먼저 필요합니다
`PPQ`를 `audio/timing.js`에서 가져옴	960이라는 값이 여러 파일에 흩어지지 않게 한 곳만 고칩니다 (S1-05와 같은 출처)
한 마디 tick 수를 인자로도 받음	박자표가 바뀌는 곡(FR-202a)에서는 마디 길이가 달라집니다. 기본값 4/4(3840)로 두고, 필요할 때 인자로 넘깁니다
템포(BPM)는 쓰지 않음	위치만 바꾸는 계산입니다. tick ↔ 초 변환은 `audio/timing.js`가 맡아 역할을 나눕니다
소수 결과를 그대로 돌려줌	반올림·스냅은 부르는 쪽(드래그 코드)이 합니다. 여기서 반올림하면 위치가 미리 틀어집니다
잘못된 값은 오류를 던짐	조용히 NaN을 넘기면 재생선이 엉뚱한 곳에 섭니다

4. 시험 (Node에서 직접 실행, 16개 통과)

확인	결과
0·960·480·3840·15360 tick이 0·0.25·0.125·1·4마디	✓
카운트인(-960 tick)이 -0.25마디	✓
5/4(4800)·3/4(2880) 마디 인자에서 1마디	✓
`TICKS_PER_BAR`가 3840	✓
tick이 NaN·Infinity·없음·문자열이면 오류	✓
마디 길이가 0·음수·NaN이면 오류	✓